Workshop: The Agentic Guardrail

Applying LKW Structural Testing to AI-Generated Code

Autonomous Drive Verification Group

March 2026

1 Introduction

As AI agents like Claude Code increasingly automate software generation, the gap between probabilistic intent and deterministic safety grows. This workshop applies the **Laski-Korel-Weyuker (LKW)** structural criteria to verify the integrity of AI-generated modules in Autonomous Driving Systems (ADS).

2 Phase 1: The Agentic Generation (The "Black Box" Problem)

2.1 Problem Statement

In the context of AI-driven development, the **Black Box Problem** arises when a Large Language Model (LLM) or code agent generates complex logic that satisfies a high-level natural language prompt but lacks a transparent structural proof of its internal data integrity. While the output may be functionally correct for "happy path" scenarios, it often contains latent data flow anomalies that are not immediately visible to the developer.

2.2 Workshop Task: Component Generation

Participants are required to use an AI agent (e.g., Claude Code) to generate a safety-critical module for an Autonomous Driving System (ADS).

- **Module:** V2X-Enabled Adaptive Cruise Control (ACC)
- **Objective:** Implement velocity control logic based on multi-source sensor and network inputs.

2.2.1 Agent Prompt Template

The following prompt should be provided to the AI agent to initiate the generation process:

"Generate a Java class `AdaptiveCruiseController` *that takes* `radarDist`, `v2xSpeedLimit`, *and* `isObstacleDetected` *as inputs. The class should calculate* `targetVelocity`.

Logic Requirements:

1. *If an obstacle is detected, set the velocity to 0.*
2. *Otherwise, the velocity should match the lower of the current speed or the V2X speed limit.*

"

2.3 Verification Challenge

Once the code is generated, the focus shifts to **Static Analysis** and **Definition-Use (DU) Mapping**. Students will analyze the agent's output to identify if any input variables (like `v2xSpeedLimit`) are defined but subsequently "terminated" or ignored by the logic branches generated by the AI.

3 Phase 2: Applying All-Definitions (The "Dead Data" Audit)

3.1 Objective

The goal of this phase is to identify instances where the AI-generated code defines variables that never influence the final actuator command or state transition.

3.2 Task: Mapping Definition-Use (DU) Pairs

Participants will perform a structural audit of the `v2xSpeedLimit` variable lifecycle.

- **Activity:** Map every point where `v2xSpeedLimit` is assigned a value (Definition) and where it is read (Use).
- **The Challenge:** AI agents frequently introduce "Safety Buffers" or "Logging Variables" that are defined at the beginning of a method but are subsequently **terminated**—where an intervening assignment or logic branch renders the initial value unreachable.
- **LKW Requirement:** Participants must verify that for every definition of a speed constraint, there exists at least one definition-clear path to a computational or predicate use.

4 Phase 3: Applying All-Uses (The "Logic Leak" Test)

4.1 Objective

This phase ensures that safety-critical definitions generated by the AI agent impact **all** intended downstream logic branches, rather than just a subset.

4.2 Task: Cross-Subsystem Impact Verification

Participants will trace the propagation of a `collisionRisk` boolean or similar safety flag.

- **Scenario:** An AI agent may correctly use `collisionRisk` to trigger a User Interface warning (*Use 1*) but fail to incorporate that same definition into the primary braking calculation (*Use 2*).
- **The Challenge:** This creates a "Logic Leak" where the system is aware of a hazard but fails to act upon it.
- **LKW Requirement:** Demonstrate that the test suite exercises definition-clear paths to **every** possible use of the critical safety definition.

5 Phase 4: Antidecomposition in Agentic Tool-Use

5.1 Objective

To prove Weyuker's Seventh Axiom by demonstrating that an AI agent's high-level success does not validate the integrity of its generated sub-components.

5.2 Task: Isolation and Stress Testing

Participants will evaluate the relationship between the primary agent logic (P) and an AI-generated utility tool (Q), such as a sensor string parser.

- **The Configuration:**
  1. The Agent (P) utilizes a generated sub-tool (Q) to interpret LiDAR data.
  2. Integration-level tests (T) pass because the Agent provides idealized input to the tool during standard operation.
- **Activity:** Participants must isolate Component Q and subject it to boundary conditions (e.g., malformed strings, null characters, or extreme values).
- **LKW Requirement:** Prove that while the test set T is adequate for the integrated plan (P), it is inadequate for the standalone component (Q), thereby necessitating modular unit testing.

6 Core LKW Criteria and AI Mapping

6.1 All-Definitions (All-Defs)

Ensures that for every definition of a variable v, there exists a definition-clear path to at least one use.

- **AI Context:** Identifies "Dead Definitions" or stub variables the AI agent created but failed to implement in the final logic.

6.2 All-Uses

Requires definition-clear paths to **every** possible use of a definition.

- **AI Context:** Ensures safety flags impact both control logic and secondary systems (e.g., logging or UI).

6.3 All-DU-Paths

The most exhaustive strategy; requires testing every simple path between a definition and a use.

- **AI Context:** Detects iterative "state-drift" where an agent's context window or loop logic fails over multiple turns.

7 Static Analysis: Automated DU-Pair Tracking

The following Python script utilizes the `ast` module to identify "Terminating Definitions"—a common anomaly in AI-generated code where a variable is redefined before its first value is consumed.

```
import ast

class DUAnalyzer(ast.NodeVisitor):
    def __init__(self):
        self.definitions = {}
        self.violations = []

    def visit_Assign(self, node):
        for target in node.targets:
            if isinstance(target, ast.Name):
                var_name = target.id
                if var_name in self.definitions:
                    self.violations.append(f"Anomaly: '{var_name}' redefined at line {node.lineno} before confirmed use.")
                self.definitions[var_name] = node.lineno
        self.generic_visit(node)

    def visit_Name(self, node):
```

```
        if isinstance(node.ctx, ast.Load) and node.id in self.definitions:
            del self.definitions[node.id]
        self.generic_visit(node)

# Example Audit
ai_code = """
def calculate_speed(radar_dist, v2x_limit):
    margin = 0.5            # Def 1 (Killed by Def 2/3)
    if v2x_limit > 100:
        margin = 1.0        # Def 2
    else:
        margin = 0.0        # Def 3
    return radar_dist + margin
"""
analyzer = DUAnalyzer()
analyzer.visit(ast.parse(ai_code))
print(analyzer.violations)

##
# Modified for file input
with open("agent_generated_code.py", "r") as source:
    tree = ast.parse(source.read())
    analyzer = DUAnalyzer()
    analyzer.visit(tree)
    # Output results...
```

Listing 1: DU-Pair Anomaly Detection Script

8 Automated LKW Structural Audit: Detecting P-Uses and C-Uses

To satisfy the **All-Uses** and **All-DU-Paths** criteria, testers must distinguish between variables that influence mathematical calculations and those that influence logical decision-making. The following Python script extends the basic static analysis to categorize **p-uses** (predicate uses) and **c-uses** (computational uses).

8.1 Python Audit Tool: LKWAnalyzer

This tool utilizes the Abstract Syntax Tree (`ast`) to traverse the AI-generated source code and identify how data definitions propagate to decision points versus arithmetic operations.

```
import ast

class LKWAnalyzer(ast.NodeVisitor):
    def __init__(self):
        self.definitions = {}  # Tracks active definitions {var: line}
        self.p_uses = []       # Tracks Predicate Uses
        self.c_uses = []       # Tracks Computational Uses
        self.terminations = [] # Tracks Terminated Definitions

```

```
    def visit_Assign(self, node):
        for target in node.targets:
            if isinstance(target, ast.Name):
                var_name = target.id
                if var_name in self.definitions:
                    self.terminations.append((var_name, node.lineno))
                self.definitions[var_name] = node.lineno
        self.generic_visit(node)

    def visit_If(self, node):
        # Identify p-uses within the 'test' expression of an 'if' block
        for name_node in ast.walk(node.test):
            if isinstance(name_node, ast.Name) and isinstance(name_node.
                ctx, ast.Load):
                self.p_uses.append((name_node.id, node.lineno))
        self.generic_visit(node)

    def visit_While(self, node):
        # Identify p-uses within the 'test' expression of a 'while' loop
        for name_node in ast.walk(node.test):
            if isinstance(name_node, ast.Name) and isinstance(name_node.
                ctx, ast.Load):
                self.p_uses.append((name_node.id, node.lineno))
        self.generic_visit(node)

    def visit_Name(self, node):
        # Capture c-uses that are not otherwise classified as p-uses
        if isinstance(node.ctx, ast.Load):
            is_p_use = any(u[0] == node.id and u[1] == node.lineno for u
                in self.p_uses)
            if not is_p_use:
                self.c_uses.append((node.id, node.lineno))
        self.generic_visit(node)

# Example execution:
# analyzer = LKWAnalyzer()
# analyzer.visit(ast.parse(source_code))
```

Listing 2: Extended Python Script for P-Use and C-Use Categorization

8.2 Mapping to Structural Adequacy

By applying this script to agent-generated code, participants can evaluate the following LKW metrics:

1. **P-Use Integrity:** If a safety-critical variable (e.g., `radar_dist`) appears in `c_uses` but is absent from `p_uses`, the AI logic may be calculating values without validating safety boundaries.

2. **Anomalous Terminations:** If the `terminations` list is non-empty, the AI has generated redundant assignments, violating the **All-Definitions** criterion and indicating a logic conflict.

3. **Path Exhaustion:** A high frequency of `p_uses` relative to the number of test cases suggests that the current test suite is likely failing the **All-Uses** criterion, as each predicate requires a True/False evaluation to validate the data flow.

9 Axiom 7: Antidecomposition in JUnit 5

The `@Nested` annotation is used here to structurally enforce the distinction between Program P and Component Q, as mandated by Weyuker's 7th Axiom.

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class ADSTestSuite {

    @Nested
    @DisplayName("Program␣P:␣System-Level␣Integration")
    class SystemTest {
        @Test
        void testSystemAdequacy() {
            // High-level integration tests that may mask component faults
            AdaptiveCruiseController acc = new AdaptiveCruiseController();
            double result = acc.calculateVelocity(50.0, 60.0, false);
            assertEquals(50.0, result);
        }
    }

    @Nested
    @DisplayName("Component␣Q:␣Isolated␣Data␣Integrity")
    class ComponentTest {
        @Test
        void testComponentAdequacy() {
            // Dedicated LKW-based unit tests for the sub-module
            SensorParser parser = new SensorParser();
            assertAll("LKW␣Inadequacy␣Checks",
                () -> assertNotNull(parser.parse("-1.0"), "Missing␣parity␣
                    handling"),
                () -> assertTrue(parser.getLastValue() >= 0, "Non-physical
                    ␣velocity")
            );
        }
    }
}
```

Listing 3: Hierarchical Antidecomposition Test Suite

Fault Category	LKW Criterion	Detected by System (P)	Detected by Unit (Q)
Redundant Code	All-Definitions	No	**Yes**
Logic Leakage	All-Uses	No	**Yes**
State Drift	All-DU-Paths	No	**Yes**
Physical Bound Error	Antidecomposition	**Partial**	**Yes**

10 Comparative Fault Analysis Summary

11 Conclusion

By applying LKW-based structural testing, developers can treat AI agents as "Black Box Plan Generators" while maintaining "White Box Mathematical Proofs" of the resulting code's safety and integrity.